

NumPy + Pandas — Easy Memory Cheat Sheet

One-line idea for every topic and sub-topic you covered.

1. NumPy Fundamentals

NumPy lets you store and process large groups of numbers super fast by using arrays instead of slow Python lists, running behind the scenes in C.

a. Why NumPy is used	Faster than lists because it uses C, not Python, under the hood.
b. Arrays vs Python lists	Lists = mixed types, slow. Arrays = one type only, fast.
c. Creating NumPy arrays	<code>zeros()</code> , <code>ones()</code> , <code>arange()</code> , <code>linspace()</code> , <code>eye()</code> , <code>random</code> — ways to build arrays.
d. Array operations	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>sum</code> , <code>mean</code> — works on the whole array at once, no loop.
e. Vectorized computation	Do math on the whole array in one line instead of looping element by element.

Remember: NumPy = 'Excel math, but super fast, no loops needed.'

2. Numerical Computing

Deeper number tools inside NumPy — applying math functions to whole arrays, auto-resizing arrays for math (broadcasting), and picking exact values out using indexing and slicing.

a. Mathematical operations	<code>sqrt</code> , <code>log</code> , <code>exp</code> , <code>sin/cos</code> — ready-made math functions (called ufuncs).
b. Broadcasting	NumPy auto-stretches a smaller array to match a bigger one during math.
c. Array indexing	Pick specific values by position, condition (True/False), or a list of positions.
d. Array slicing	Grab a continuous chunk (<code>start:stop:step</code>). Slices are VIEWS — editing one edits the original!
e. Multi-dimensional arrays	<code>shape</code> / <code>ndim</code> / <code>size</code> tell you the array's dimensions: rows, columns, and depth.

Remember: Broadcasting = stretching. Slicing = a live window, not a copy.

3. Pandas Fundamentals

Pandas turns raw data into labeled tables like Excel or SQL, built on top of NumPy, so every value has a meaningful row and column name attached.

a. Series	One labeled column: values + an index (label) attached to each value.
b. DataFrames	A full table = many Series (columns) sharing the same row labels.
c. Loading datasets	<code>read_csv()</code> , <code>read_excel()</code> , <code>read_json()</code> — bring outside files into pandas.
d. Inspecting data	<code>head()</code> , <code>info()</code> , <code>shape</code> , <code>dtypes</code> — quick first check of any new dataset.
e. Data exploration	<code>describe()</code> , <code>value_counts()</code> , <code>corr()</code> — understand patterns before analyzing.

Remember: Series = one labeled column. DataFrame = the whole labeled table.

4. Data Manipulation

The everyday actions on a table: filtering rows by condition, sorting by value, grouping by category, summarizing with aggregations, and cleaning up missing or empty values.

a. Filtering data	Keep only rows matching a condition, e.g. <code>df[df['Age'] > 25]</code> .
b. Sorting data	<code>sort_values()</code> — reorder rows by a column, ascending or descending.
c. Grouping data	<code>groupby()</code> — split data into categories, like Excel's pivot table groups.
d. Aggregations	<code>agg()</code> / <code>sum()</code> / <code>mean()</code> — shrink a group down into one summary number.
e. Handling missing values	<code>isnull()</code> , <code>dropna()</code> , <code>fillna()</code> — find and deal with empty (NaN) values.

Remember the order: Clean -> Filter -> Sort -> Group -> Aggregate.

5. Data Analysis Workflows

The full mindset for analyzing real data: inspect it, summarize it, transform it, explore which columns matter, and think critically before trusting any conclusion you find.

a. Dataset inspection	First mechanical check: <code>shape</code> , <code>dtypes</code> , missing values (<code>info()</code>).
b. Data summarization	Turn raw rows into meaningful averages, counts, and patterns.
c. Data transformations	<code>apply()</code> , <code>map()</code> , <code>astype()</code> — reshape or recompute columns into a useful form.
d. Feature exploration	Check if a specific column actually matters for your question.
e. Analytical thinking	Don't just trust a number — check the sample size before believing it!

Remember: Inspect -> Summarize -> Transform -> Explore -> Think Critically.

**Big Picture: NumPy (fast numbers) -> Pandas (labeled tables) -> Manipulation (clean/filter/group)
-> Workflow (think like an analyst)**